Newton School of Technology

11

Structural Design Patterns - Adapter and Composite Pattern

by Priya Sharma

What are Structural Patterns ?

Structural patterns focus on how objects and classes are composed to form larger, more flexible structures.They focus on **composition** rather than creation.

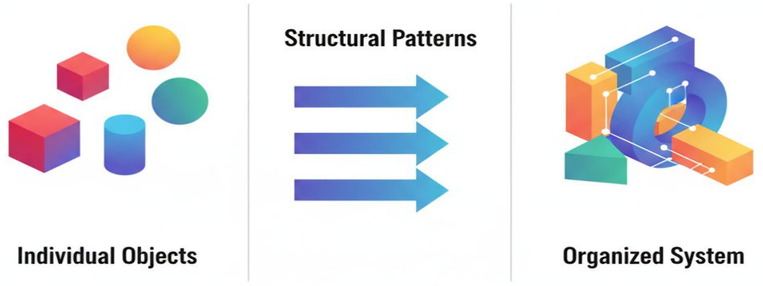
Key Focus Areas:-

Relationships between objects

Composition and assembly

Building complex systems from simple parts

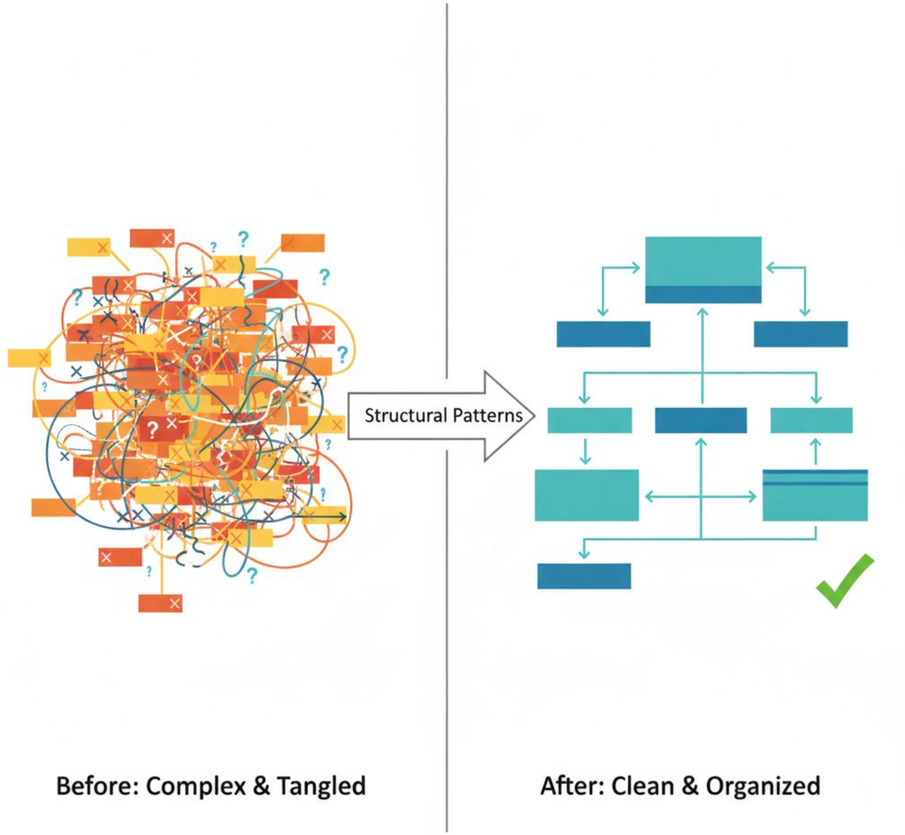
Maintaining flexibility and efficiency



Think of it like: You have the building blocks (objects) - now how do you connect them to create something powerful?

Why Do We Need Structural Patterns?

- To **reduce complexity**
- To **reuse existing code**
- To **make systems flexible**
- To **avoid tight coupling** between classes
- To **simplify system structure**



Real World Example: E-Commerce Platform

- Individual blocks = **Objects**

  Example: A User class, a Product class

- How you connect them = **Structural patterns**

  Example: How User relates to Orders

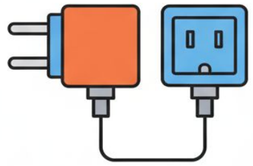
- The final structure = **Your system**

  Example: A working e-commerce platform

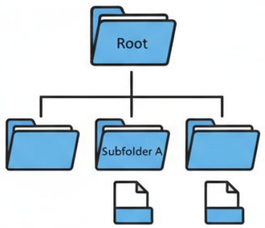
The Structural Pattern Family

1. Adapter

- Makes incompatible interfaces work together
- Like a power adapter for travel



Adapter Pattern



Composite Pattern

2. Composite

- Organizes objects into tree structures
- Like a file system (folders within folders)

Other Structural Design patterns include: **Bridge**, **Decorator**, **Facade** and **Proxy**.

The Problem: Interface Mismatch

You're building an e-commerce site. You've written all your payment code expecting a **PaymentProcessor** interface with a pay() method.

Plot Twist: Your company decides to use a third-party payment library(e.g - razorpay), but it has:

- Different method names: makePayment() instead of pay()
- Different parameters: Takes valueInDollars instead of amount

The Problem: Interface Mismatch

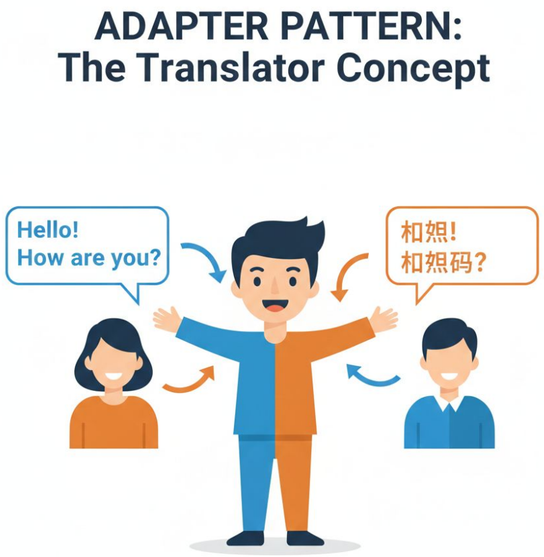
- **Bad Option:** Rewrite your entire codebase to match the third-party API
- **Worse Option:** Directly use the third-party API everywhere (coupling nightmare)
- **Good Option:** Use the Adapter Pattern!

What is Adapter Pattern ?

- The Adapter pattern creates a "wrapper" that translates one interface into another, making incompatible interfaces work together.

- Think of it as a translator who sits between two people speaking different languages. They understand both sides and convert messages back and forth.

- **Core Principle:** Don't change existing code; wrap it instead!



Adapter in Daily Life

Example 1: Travel Power Adapter

- **Your Device:** Indian 3-pin plug (Target Interface)
- **Foreign Socket:** US 2-pin socket (Adaptee)
- **Power Adapter:** Converts 3-pin to 2-pin (Adapter)
- **Result:** Your device works in the US!

Adapter in Daily Life

Example 2: USB-C to HDMI Dongle

- **Your Laptop:** Only has USB-C ports (Target)
- **Projector:** Only has HDMI input (Adaptee)
- **Dongle:** Converts USB-C to HDMI (Adapter)
- **Result:** You can present on the projector!

Example: Payment Integration

Your e-commerce system has a standard payment interface that all payment processors must follow.

Your System's Interface (Target)

```
interface PaymentProcessor {
  pay(amount: number): void;
}
```

Third-Party Library You Must Use (Adaptee)

```
class ThirdPartyPaymentService {
  makePayment(valueInDollars: number): void {
    console.log(`Paid $${valueInDollars} via External API`);
  }
}
```

Example: Payment Integration

The Problem:

- Your code expects **pay(amount)**
- Third-party provides **makePayment(valueInDollars)**
- Method names don't match!
- You CANNOT modify the third-party library

The Challenge: How do you make them work together without rewriting your entire codebase?

Example: Payment Integration

The Adapter Solution

Step 1: Create the Adapter Class

```
class PaymentAdapter implements PaymentProcessor {
  constructor(private service: ThirdPartyPaymentService) {}

  pay(amount: number): void {
    // Translation happens here!
    this.service.makePayment(amount);
  }
}
```

Example: Payment Integration

Step 2: Use It Seamlessly

```
// Create the third-party service
const service = new ThirdPartyPaymentService();

// Wrap it with adapter
const processor: PaymentProcessor = new PaymentAdapter(service);

// Your code works without knowing about the third-party details!
processor.pay(500);
// Output: Paid $500 via External API
```

Your entire codebase still uses `PaymentProcessor` interface.
Only one small adapter class knows about the third-party API!

Example: Payment Integration

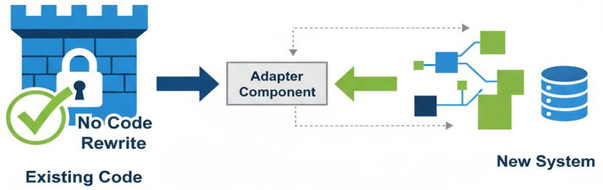
What's Happening:

- Adapter implements the **PaymentProcessor** interface (what your system expects)
- Internally, it holds a reference to **ThirdPartyPaymentService**
- The **pay()** method translates calls to **makePayment()**

Adapter Pattern: Benefits

Benefit 1: No Code Rewrite

- Your existing code remains unchanged
- Only create one small adapter class
- Saves weeks of refactoring work!



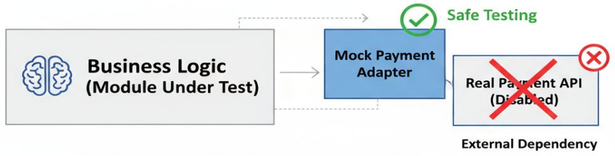
Benefit 2: Easy to Switch Vendors

- Third-party API changed? Update only the adapter!
- Want to switch payment providers? Create a new adapter!
- Your business logic stays untouched

Adapter Pattern: Benefits

Benefit 3: Testing Made Easy

- Create a MockPaymentAdapter for testing
- No need to hit real payment APIs during tests
- Full control over test scenarios



Benefit 4: Open-Closed Principle

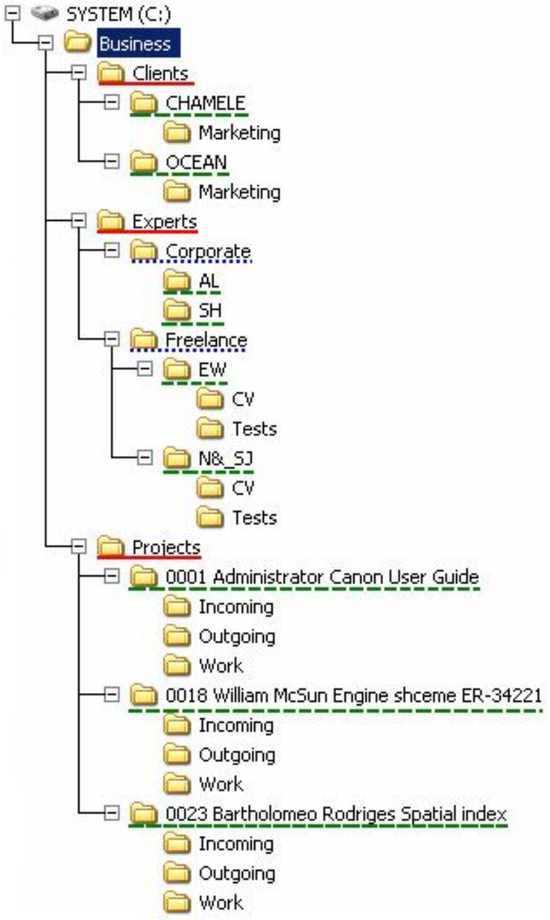
- System is open for extension (add new adapters)
- Closed for modification (existing code unchanged)

The Problem: Tree Like Structure

You're building a file manager application.
You have:

- **Files:** Individual items with a size
- **Folders:** Containers that hold files OR other folders

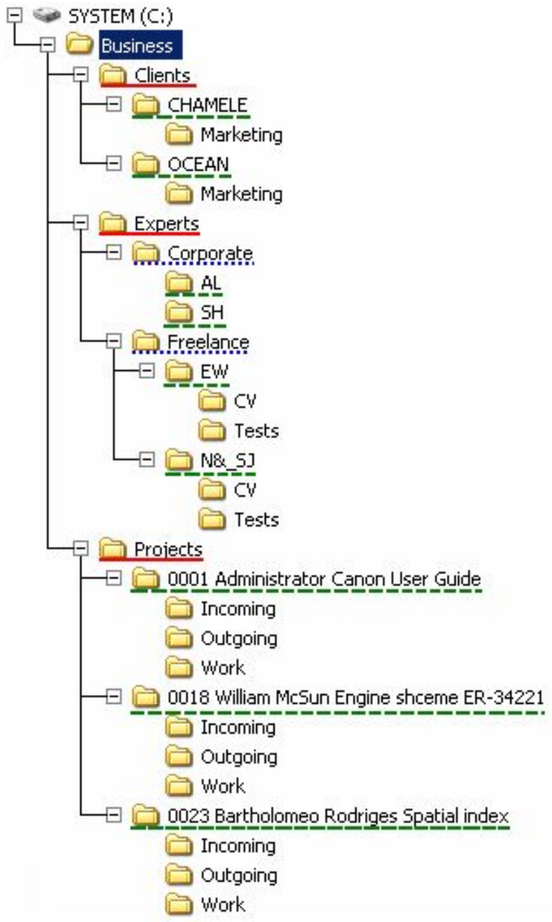
Your Challenge: Calculate the total size of a folder (including all nested folders and files).



The Problem: Tree Like Structure

Traditional Approach (Messy):

```
function getTotalSize(item: any): number {
  if (item.isFile) {
    return item.size;
  } else if (item.isFolder) {
    let total = 0;
    for (let child of item.children) {
      if (child.isFile) {
        total += child.size;
      } else if (child.isFolder) {
        // Recursion needed... this gets complicated!
      }
    }
    return total;
  }
}
```



What is the Composite Pattern ?

The Composite pattern lets you compose objects into tree structures and treat individual objects and compositions uniformly.

Make both "single items" (leaves) and "containers" (composites) look the same to the outside world. The client doesn't need to care whether it's dealing with one or many.

Key Insight: Both File and Folder implement the same interface with the same methods. The client just calls the method without checking types!

What is the Composite Pattern ?

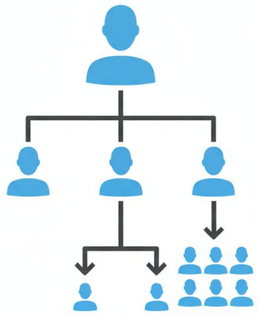
Pattern Structure:

- **Component Interface:** Common interface for all objects (getSize, delete, etc.)
- **Leaf:** Individual object with no children (File)
- **Composite:** Container that holds other components (Folder)
- **Client:** Code that works with the component interface

Core Principle: Treat part and whole uniformly!

Composite in Daily Life

Example 1: Organization Chart



- **Employee (Leaf):** Individual contributor, no subordinates
- **Manager (Composite):** Has subordinates (who can also be managers)
- **Common Operation:** Calculate total salary
  - Employee: Just their salary
  - Manager: Their salary + all subordinates' salaries (recursive)

Composite in Daily Life

Example 2: Restaurant Menu

- **Menu Item (Leaf):** Pizza ₹299, Burger ₹150
- **Sub-Menu (Composite):** "Combo Meals" contains multiple items
- **Common Operation:** Calculate total price
  - Item: Just its price
  - Sub-menu: Sum of all items inside

Example: File System

Building a file manager that can calculate the total size of files and folders.

```
interface FileSystemItem {
  getSize(): number;
}
```

Both Files and Folders must implement this interface!

Example: File System

Why This Interface?

- Defines what operations all items must support
- Client code can call getSize() on anything without type checking
- Makes the system extensible (add new types easily)

```
// Client code - works for BOTH files and folders!
function printSize(item: FileSystemItem): void {
  console.log(`Size: ${item.getSize()} bytes`);
}
```

Composite Pattern: Implementation

Step 1: The Leaf - File

Concept: A File is a "leaf" - it has no children and represents an individual item.

```
class FileItem implements FileSystemItem {
  constructor(
    private name: string,
    private size: number
  ) {}

  getSize(): number {
    return this.size;  // Just return its own size
  }

  getName(): string {
    return this.name;
  }
}
```

Composite Pattern: Implementation

Characteristics of a Leaf:

- Has no children
- Implements the component interface
- Performs operations only on itself
- Simple, straightforward behavior

Example Usage:

```
const file1 = new FileItem("photo.jpg", 2048);
const file2 = new FileItem("document.pdf", 5120);

console.log(file1.getSize());  // Output: 2048
console.log(file2.getSize());  // Output: 5120
```

Composite Pattern: Implementation

Newton School of Technology

Step 2: The Composite - Folder

```
class Folder implements FileSystemItem {
  private children: FileSystemItem[] = [];

  constructor(private name: string) {}

  add(item: FileSystemItem): void {
    this.children.push(item);
  }

  remove(item: FileSystemItem): void {
    const index = this.children.indexOf(item);
    if (index > -1) {
      this.children.splice(index, 1);
    }
  }

  getSize(): number {
    // Recursively calculate size of all children!
    return this.children.reduce(
      (total, child) => total + child.getSize(),
      0
    );
  }

  getName(): string {
    return this.name;
  }
}
```

Composite Pattern: Implementation

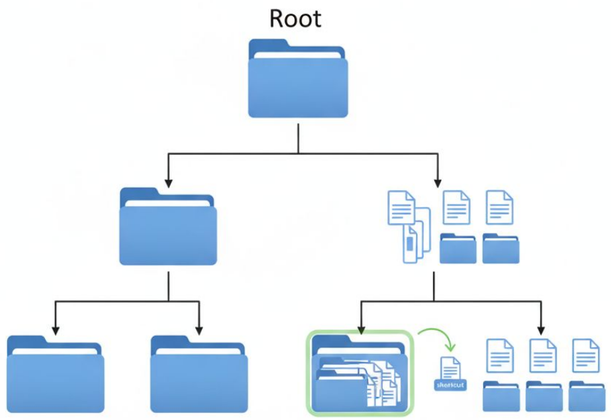
Putting it all together:

```
// Create files (leaves)
const file1 = new FileItem("photo.jpg", 2048);
const file2 = new FileItem("document.pdf", 5120);
const file3 = new FileItem("video.mp4", 10240);

// Create folders (composites)
const documentsFolder = new Folder("Documents");
const mediaFolder = new Folder("Media");
const rootFolder = new Folder("Root");

// Build the tree
documentsFolder.add(file2);  // Documents contains document.pdf

mediaFolder.add(file1);      // Media contains photo.jpg
mediaFolder.add(file3);      // Media contains video.mp4

rootFolder.add(documentsFolder);  // Root contains Documents folder
rootFolder.add(mediaFolder);      // Root contains Media folder

// Calculate sizes - same method for everything!
console.log(`Document size: ${file2.getSize()}`);
// Output: Document size: 5120

console.log(`Media folder size: ${mediaFolder.getSize()}`);
// Output: Media folder size: 12288 (2048 + 10240)

console.log(`Total root size: ${rootFolder.getSize()}`);
// Output: Total root size: 17408 (5120 + 2048 + 10240)
```

Composite Pattern: Benefits

Benefit 1: Uniform Treatment

- Treat single objects and collections the same way
- No type checking (if file... else if folder...)
- Clean, readable code



Benefit 2: Easy to Add New Types

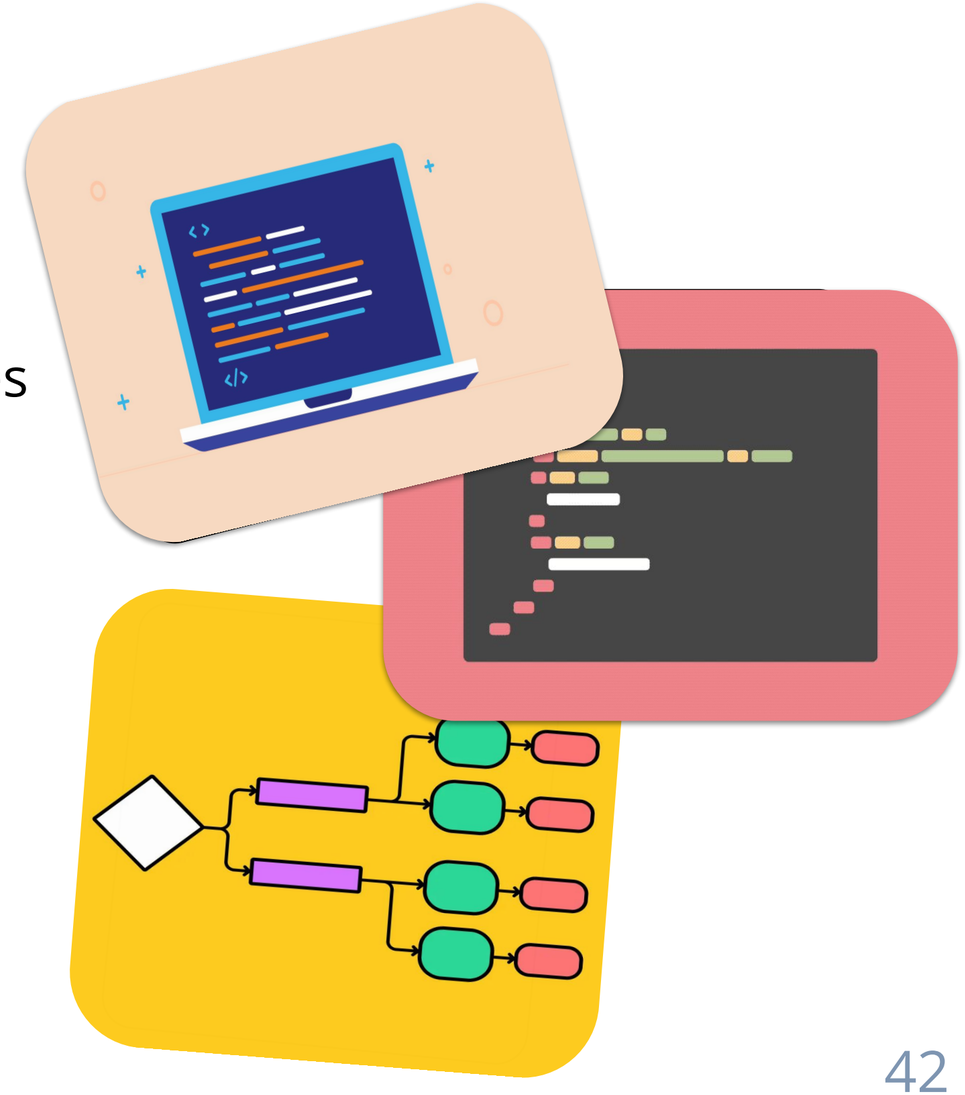
- Want to add "Shortcut" or "SymbolicLink"?
- Just implement FileSystemItem interface
- Existing code works without changes

Composite Pattern: Benefits

Benefit 3: Recursive Operations

- Complex operations become simple
- getSize() automatically handles any nesting depth
- Delete a folder? Automatically deletes everything inside

Coding Questions

Summary

- **Structural patterns** focus on how classes and objects are composed to form larger structures.
- **Adapter** is your go-to for making incompatible interfaces work together without changing their source code.
- **Composite** allows you to treat "parts" and "wholes" exactly the same, which is perfect for hierarchical data.
- Both patterns strongly support the **Open-Closed Principle**.